

✓ COHORT ANALYSIS

CosmeticWorld is a global retail chain specializing in cosmetics and personal care products. Over four years of operation (2020–2023), the company has recorded more than 50,000 transactions across its sales network.

Problem Statement

- What was each customer's first purchase date, and what were their retention rates throughout 2022?
- Which segment had better retention rates, the Corporate or the Consumer?
- Which Annual Income group contributes the most sustainable profitability to the business?

Datasets Used

- EcomSales.csv: Sales transaction data.
- Customer.csv: Customer information.
- Product.csv: Product information.
- Region.csv: Geographic and regional information.

✓ I. Setup & Data Ingestion

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime

pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.expand_frame_repr', None)
```

```
from google.colab import drive
drive.mount('/content/drive') # Mount Google Drive to access files
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
df_sales = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/EcomSales.csv')
df_customers = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Customer.csv')
df_products = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Product.csv')
df_regions = pd.read_csv('/content/drive/MyDrive/Analytics Report/data/Region.csv')
```

✓ II. Data Cleaning & Pre-processing

```
# Declare a function to check the dataframe.

def check_df(df):
    print('=' * 50)
    print('DATAFRAME INFORMATION')
    print('=' * 50)
    # Print concise summary of the DataFrame
    df.info()

    print('=' * 50)
    print('NULL VALUES')
    print('=' * 50)
    # Count null values per column
    print(df.isna().sum())

    print('=' * 50)
    print('DUPLICATE VALUES')
    print('=' * 50)
    # Count duplicate rows
    print(f" Number of duplicates: {df.duplicated().sum()}")

    print('=' * 50)
    print('DESCRIPTIVE STATISTICS')
    print('=' * 50)
    # Generate descriptive statistics
    print(df.describe().T)
```

▼ Inspect df_sales

```
check_df(df_sales) # Apply the check_df function to the sales DataFrame
```

```
# Create a box plot to visualize the distribution and outliers of 'Quantity'
sns.boxplot(data = df_sales, x = 'Quantity')
```

```
# Create a box plot to visualize the distribution and outliers of 'Sales'
sns.boxplot(data = df_sales, x = 'Sales')
```

```
# Create a box plot to visualize the distribution and outliers of 'Profit'
sns.boxplot(data = df_sales, x = 'Profit')
```

Note: Sales, Quantity, Profit have outliers

▼ Inspect df_customers

```
# Apply the check_df function to the customers DataFrame
check_df(df_customers)
```

```
# Create a box plot to visualize the distribution and outliers of 'AnnualIncome'
sns.boxplot(data = df_customers, x = 'AnnualIncome')
```

Note: `Gender` has null values and `AnnualIncome` has outliers

Orders with revenue identified as outliers are likely orders from potential customers, a minority who require more special attention than the majority.

High revenue also means a large number of products in a single order (in the `Quantity` column), and `Profit` also fluctuates significantly, leading to the result of becoming outliers.

Outliers in the `AnnualIncome` of `df_customers` may include potential customers.

✓ Inspect `df_products`

```
check_df(df_products) # Apply the check_df function to the products DataFrame
```

✓ Inspect `df_regions`

```
check_df(df_regions) # Apply the check_df function to the regions DataFrame
```

Handling null values

```
df_customers['Gender'].fillna('Undefined', inplace = True) # Fill null values in 'Gender' column with 'Undefined'
df_customers[df_customers['Gender'] == 'Undefined'].head() # Display first few rows where 'Gender' is 'Undefined'
```

Check for unusual codes in `df_sales`

```
import re
```

```
# Define the pattern for ProductCode (1 uppercase letter followed by 6 digits, e.g., P000001)
pattern = r'^[A-Z]\d{6}$'
```

```
# Find data that does not conform to the above format
non_conforming_product_codes = df_sales[~df_sales['ProductCode'].str.fullmatch(pattern, na=False)]
```

```
print(non_conforming_product_codes) # Display non-conforming product codes
```

```
# Define the pattern for RegionCode (1 uppercase letter followed by 4 digits, e.g., R0001)
pattern = r'^[A-Z]\d{4}$'
```

```
# Find data that does not conform to the above format
non_conforming_region_codes = df_sales[~df_sales['RegionCode'].str.fullmatch(pattern, na=False)]
```

```
print(non_conforming_region_codes) # Display non-conforming region codes
```

For data with different encoding formats than those in the dimension table, it is necessary to check with relevant sources or confirm with stakeholders to verify before making any modifications to these unusual product and region codes.

```
# Change data type

df_sales['OrderDate'] = df_sales['OrderDate'].astype('datetime64[ns]') # Convert 'OrderDate' column to datetime objects
```

▼ III. Cohort Analysis

▼ 3.1. Determine the Acquisition Month for each customer and calculate the retention rate for 2022.

```
# Get the Acquisition Month (first order month) for each customer

# Group by CustomerID and find the minimum OrderDate
df_first_time = df_sales.groupby(by = 'CustomerID', as_index = False).agg(first_order_date = ('OrderDate', 'min'))
df_first_time['first_month'] = df_first_time['first_order_date'].dt.to_period('M')

df_first_time # Display the DataFrame with customer's first order month
```

```
# Create a DataFrame with unique CustomerID and OrderDate pairs
df = df_sales[['CustomerID', 'OrderDate']].drop_duplicates(ignore_index = True)

# Merge with the first order date information
df_pre_cohort = df.merge(df_first_time, on = 'CustomerID', how = 'inner')

df_pre_cohort
```

```
# Filter data for customers acquired in 2022 and orders made in 2022
df_pre_cohort = df_pre_cohort[(df_pre_cohort['first_month'].dt.year == 2022) & (df_pre_cohort['OrderDate'].dt.year == 2022)]
df_pre_cohort # Display the filtered pre-cohort DataFrame
```

```
# Calculate the number of months since the first order
df_pre_cohort['month_after'] = (df_pre_cohort['OrderDate'].dt.year - df_pre_cohort['first_order_date'].dt.year) * 12 + (df_pre_cohort['OrderDate'].dt.month - df_pre_cohort['first_order_date'].dt.month)
df_pre_cohort # Display the DataFrame with 'month_after' column
```

```
# Calculate the number of months since the first order
df_cohort = df_pre_cohort.pivot_table(index = 'first_month',
                                      columns = 'month_after',
                                      values = 'CustomerID',
                                      aggfunc = 'nunique')

df_cohort # Display the cohort DataFrame
```

```
empty_column = pd.Series([0] * len(df_cohort), name='CC') # Create an empty Series for 'CC' column (Cohort Count)

df_cohort.insert(loc=0, column='CC', value=empty_column) # Insert 'CC' column at the beginning of the cohort DataFrame

cohort_size_monthly_new = df_cohort.iloc[:, 1] # Get the cohort size (number of unique customers in the first month)
```

```
retention_matrix_monthly_new = df_cohort.divide(cohort_size_monthly_new, axis=0) # Calculate retention rates by dividing by cohort size

retention_matrix_monthly_new # Display the retention matrix
```

```
plt.figure(figsize= (15,8)) # Set the figure size for the heatmap

sns.heatmap(data = retention_matrix_monthly_new # Plot the retention matrix
            ,annot=True # Annotate cells with the retention percentage
            ,cmap='YlGnBu' # Use 'YlGnBu' colormap
            ,fmt = '.0%') # Format annotations as percentages without decimals

for i in range(len(df_cohort)): # Loop through each row of the cohort DataFrame
    plt.text(0.5, i + 0.5, '{:,.0f}'.format(df_cohort.iloc[i, 1]) # Add text for cohort size (CC column)
            , ha='center' # Center horizontally
            , va='center' # Center vertically
            , color = 'red') # Set text color to red

plt.xlabel('Cohort Index') # Set x-axis label
plt.ylabel('First Order Month') # Set y-axis label
plt.title('Cohort Analysis') # Set plot title

plt.show() # Display the plot
```

3.2. Compare retention rates between Corporate and Consumer segments.

```
# Cohort Analysis by Corporate segment
df_sales_corporate = df_sales[df_sales['Segment'] == 'Corporate'] # Filter sales data for Corporate segment

df_first_time_corporate = df_sales_corporate.groupby(by = 'CustomerID', as_index = False).agg(first_order_date = ('OrderDate', 'min')) # Find first order date for each corporate customer
df_first_time_corporate['first_month'] = df_first_time_corporate['first_order_date'].dt.to_period('M') # Extract first order month

df_corporate = df_sales_corporate[['CustomerID', 'OrderDate']].drop_duplicates(ignore_index = True) # Get unique customer and order date pairs for corporate
df_pre_cohort_corporate = df_corporate.merge(df_first_time_corporate, on = 'CustomerID', how = 'inner') # Merge with first order date info

df_pre_cohort_corporate = df_pre_cohort_corporate[(df_pre_cohort_corporate['first_month'].dt.year == 2022) & (df_pre_cohort_corporate['OrderDate'].dt.year == 2022)] # Filter for 2022 data

# Calculate months after first order
df_pre_cohort_corporate['month_after'] = (df_pre_cohort_corporate['OrderDate'].dt.year - df_pre_cohort_corporate['first_order_date'].dt.year) * 12 + (df_pre_cohort_corporate['OrderDate'].dt.month - df_pre_cohort_corporate['first_order_date'].dt.month)

df_cohort_corporate = df_pre_cohort_corporate.pivot_table(index = 'first_month', columns = 'month_after', values = 'CustomerID', aggfunc = 'nunique') # Create cohort pivot table

empty_column_corporate = pd.Series([0] * len(df_cohort_corporate), name='CC') # Create empty Series for 'CC' column
df_cohort_corporate.insert(loc=0, column='CC', value=empty_column_corporate) # Insert 'CC' column
cohort_size_monthly_corporate = df_cohort_corporate.iloc[:, 1] # Get cohort sizes
retention_matrix_monthly_corporate = df_cohort_corporate.divide(cohort_size_monthly_corporate, axis=0) # Calculate retention matrix

plt.figure(figsize= (15,8)) # Set figure size
sns.heatmap(data = retention_matrix_monthly_corporate # Plot retention heatmap
            ,annot=True
            ,cmap='YlGnBu'
            ,fmt = '.0%')

for i in range(len(df_cohort_corporate)): # Add cohort size labels
```

```

plt.text(0.5, i + 0.5, '{:,.0f}'.format(df_cohort_corporate.iloc[i, 1])
        , ha='center'
        , va='center'
        , color = 'red')

plt.xlabel('Cohort Index') # Set x-axis label
plt.ylabel('First Order Month') # Set y-axis label
plt.title('Cohort Analysis by Corporate') # Set title
plt.show() # Display plot

```

```

# Cohort Analysis by Consumer segment

df_sales_consumer = df_sales[df_sales['Segment'] == 'Consumer'] # Filter sales data for Consumer segment

df_first_time_consumer = df_sales_consumer.groupby(by = 'CustomerID', as_index = False).agg(first_order_date = ('OrderDate', 'min')) # Find first order date for each consumer customer
df_first_time_consumer['first_month'] = df_first_time_consumer['first_order_date'].dt.to_period('M') # Extract first order month

df_consumer = df_sales_consumer[['CustomerID','OrderDate']].drop_duplicates(ignore_index = True) # Get unique customer and order date pairs for consumer
df_pre_cohort_consumer = df_consumer.merge(df_first_time_consumer, on = 'CustomerID', how = 'inner') # Merge with first order date info

df_pre_cohort_consumer = df_pre_cohort_consumer[(df_pre_cohort_consumer['first_month'].dt.year == 2022) & (df_pre_cohort_consumer['OrderDate'].dt.year == 2022)] # Filter for 2022 data

# Calculate months after first order
df_pre_cohort_consumer['month_after'] = (df_pre_cohort_consumer['OrderDate'].dt.year - df_pre_cohort_consumer['first_order_date'].dt.year) * 12 + (df_pre_cohort_consumer['OrderDate'].dt.mo

df_cohort_consumer = df_pre_cohort_consumer.pivot_table(index = 'first_month', columns = 'month_after', values = 'CustomerID', aggfunc = 'nunique') # Create cohort pivot table

empty_column_consumer = pd.Series([0] * len(df_cohort_consumer), name='CC') # Create empty Series for 'CC' column
df_cohort_consumer.insert(loc=0, column='CC', value=empty_column_consumer) # Insert 'CC' column
cohort_size_monthly_consumer = df_cohort_consumer.iloc[:, 1] # Get cohort sizes
retention_matrix_monthly_consumer = df_cohort_consumer.divide(cohort_size_monthly_consumer, axis=0) # Calculate retention matrix

plt.figure(figsize= (15,8)) # Set figure size
sns.heatmap(data = retention_matrix_monthly_consumer # Plot retention heatmap
            ,annot=True
            ,cmap='YlGnBu'
            ,fmt = '.0%')

for i in range(len(df_cohort_consumer)): # Add cohort size labels
    plt.text(0.5, i + 0.5, '{:,.0f}'.format(df_cohort_consumer.iloc[i, 1])
            , ha='center'
            , va='center'
            , color = 'red')

plt.xlabel('Cohort Index') # Set x-axis label
plt.ylabel('First Order Month') # Set y-axis label
plt.title('Cohort Analysis by Consumer') # Set title
plt.show() # Display plot

```

3.3. Which Annual Income group contributes the most sustainable profitability?

```

# Group by 'Occupation' to find average 'AnnualIncome' and sort in descending order
df_avg_annualincome = df_customers.groupby(by = 'Occupation', as_index = False).agg(avg_annualincome = ('AnnualIncome', 'mean')).sort_values(by = 'avg_annualincome', ascending = False)
df_avg_annualincome # Display the DataFrame

def plot_profitable_cohort_by_occupation(df_sales, df_customers, occupation, year=2020):
    # Merge sales and customer data
    df_merged = pd.merge(df_sales, df_customers[['CustomerID', 'Occupation', 'AnnualIncome']], on='CustomerID', how='inner')

    # Filter for the specified occupation
    df_occupation = df_merged[df_merged['Occupation'] == occupation].copy() # Filter for a specific occupation

    # Filter for transactions with positive profit
    df_profitable_sales = df_occupation[df_occupation['Profit'] > 0].copy() # Keep only sales with positive profit

    # Get unique CustomerID and OrderDate for profitable sales
    df_unique_profitable_purchases = df_profitable_sales[['CustomerID', 'OrderDate']].drop_duplicates(ignore_index=True) # Get unique customer-order date for profitable purchases

    # Determine the first profitable order date for each customer
    df_first_profitable_order = df_unique_profitable_purchases.groupby('CustomerID', as_index=False).agg(
        first_profitable_order_date=('OrderDate', 'min')
    )
    df_first_profitable_order['first_profitable_month'] = df_first_profitable_order['first_profitable_order_date'].dt.to_period('M') # Extract first profitable month

    # Merge back to get the pre-cohort data with first profitable month
    df_pre_cohort_profitable = pd.merge(df_unique_profitable_purchases, df_first_profitable_order, on='CustomerID', how='inner') # Merge with first profitable order information

    # Filter data for the specified year
    df_pre_cohort_profitable = df_pre_cohort_profitable[
        (df_pre_cohort_profitable['first_profitable_month'].dt.year == year) &
        (df_pre_cohort_profitable['OrderDate'].dt.year == year)
    ] # Filter data for a specific year

    # Calculate month_after (cohort index)
    df_pre_cohort_profitable['month_after'] = (
        (df_pre_cohort_profitable['OrderDate'].dt.year - df_pre_cohort_profitable['first_profitable_order_date'].dt.year) * 12 +
        (df_pre_cohort_profitable['OrderDate'].dt.month - df_pre_cohort_profitable['first_profitable_order_date'].dt.month)
    ) # Calculate months after the first profitable order

    # Create the cohort pivot table
    df_cohort_profitable = df_pre_cohort_profitable.pivot_table(
        index='first_profitable_month',
        columns='month_after',
        values='CustomerID',
        aggfunc='nunique'
    ) # Create a pivot table for profitable cohort analysis

    # Cohort size
    cohort_sizes = df_cohort_profitable.iloc[:, 0].copy() # Get the size of each cohort

    # Retention %
    retention_matrix = df_cohort_profitable.divide(
        cohort_sizes,
        axis=0
    ) # Calculate the retention matrix

    # Labels %

```

```

labels = retention_matrix.applymap(
    lambda x: f'{x*100:.0f}%' if pd.notna(x) else ''
) # Format retention percentages for annotation

# Plot
fig, ax = plt.subplots(figsize=(15, 8)) # Create a figure and axes for the plot

sns.heatmap(
    retention_matrix,
    annot=labels,
    fmt='',
    cmap='YlGnBu',
    ax=ax
) # Create the heatmap

ax.set_xlim(-1, retention_matrix.shape[1]) # Set x-axis limits

# CC column
for y, value in enumerate(cohort_sizes):
    ax.text(
        -0.5,
        y + 0.5,
        f'{int(value)}',
        ha='center',
        va='center',
        color='red',
        fontsize=11
    ) # Add cohort size labels to the plot

xticks = [-0.5] + [i + 0.5 for i in range(retention_matrix.shape[1])]
xticklabels = ['CC'] + list(retention_matrix.columns)

ax.set_xticks(xticks)
ax.set_xticklabels(xticklabels)

ax.set_xlabel('Cohort Index') # Set x-axis label
ax.set_ylabel('First Order Month') # Set y-axis label
ax.set_title(f'Cohort Analysis by {occupation}') # Set plot title

plt.show() # Display the plot

```

```

# Plot profitable cohort analysis for 'Management' occupation in 2022
plot_profitable_cohort_by_occupation(df_sales, df_customers, occupation='Management', year=2022)

```

```

# Plot profitable cohort analysis for 'Professional' occupation in 2022
plot_profitable_cohort_by_occupation(df_sales, df_customers, occupation='Professional', year=2022)

```

```

# Plot profitable cohort analysis for 'Skilled Manual' occupation in 2022
plot_profitable_cohort_by_occupation(df_sales, df_customers, occupation='Skilled Manual', year=2022)

```

```

# Plot profitable cohort analysis for 'Clerical' occupation in 2022
plot_profitable_cohort_by_occupation(df_sales, df_customers, occupation='Clerical', year=2022)

```



```
# Plot profitable cohort analysis for 'Manual' occupation in 2022
plot_profitable_cohort_by_occupation(df_sales, df_customers, occupation='Manual', year=2022)
```

✓ IV. Conclusion

4.1. Customer Retention Rate Assessment in 2022

In 2020, CosmeticWorld is attracting new customers quite well, but the customer retention rate is low, mostly only 1-2%. Cohort analysis shows that most customers make a one-time purchase and do not return in subsequent months of the year. In the short term, the business should focus resources on shortening the time between the first and second purchases, researching and implementing loyalty programs/campaigns, and improving after-sales care.

4.2. Comparing Customer Retention Rates between Corporate and Consumer Segments (2022)

Both groups maintained retention rates mainly between 1%–4%. Corporate had higher retention rates than Consumer at certain periods:

- June 2022 reached 6% in the second month.
- May 2022 and July 2022 reached 5% at various times.

Overall, retention rates are low, and a sufficiently large loyal customer base has not yet been formed.

In the short term, budget should prioritize customer retention for the Consumer group to maximize repeat revenue (this group buys smaller quantities and has potential for more frequent returns). Shift focus from solely acquiring new customers to increasing second-time and recurring purchases in both segments. Additionally, re-evaluate the design of care and recurring purchase programs for Corporate customers to identify areas for improving the stability of B2B revenue.

4.3. Which Annual Income group provides the most sustainable profitability (2022)

The customer group with an average Annual Income of ~\$31,000, corresponding to the Clerical occupation, is the group that brings the most sustainable profitability.

The contribution rate to profit over each month for the Clerical group appears more frequently and extends sustainably to the 9th and 10th months with a repeat rate of 3% to 9%.

Contrary to expectations, higher-income groups (Management and Professional) are very sparse and inconsistent (only around 1% - 3%).

It is advisable to focus the customer care budget, send regular offers, and build a loyalty card program for customers with an average income of ~\$31,000 to maintain purchase frequency.

Design product packages or pricing policies, combos with practical payment levels, suitable for the ~\$31,000 income level to stimulate regular monthly purchases.

Additionally, it is necessary to survey high-income groups, review after-sales service quality, or personalize the experience for the Premium segment to increase purchase likelihood, avoiding wasted initial customer acquisition costs.

Furthermore, it is also necessary to analyze customers with negative profit orders to find ways to optimize order costs and increase the number of customers contributing to monthly profit.

